NumPy Cheat Sheet

A quick reference for the numpy you need to fill in the functions marked `pass`. It covers only what you have to write yourself and explains how each tool works. The examples use unrelated toy data, so you will need to apply the ideas to the exercises yourself. Every snippet runs on its own after the import below.

```
import numpy as np
rng = np.random.default_rng()   # random number generator; the exercise files
already create this for you
```

1. Arrays and shapes

A numpy array is a grid of numbers, all of the same type. Its **shape** tells you how many numbers there are along each dimension.

```
heights = np.array([[170.0, 172.5, 169.0],
                    [181.0, 179.5, 180.0]])   # 2 rows, 3 columns
print(heights.shape)      # (2, 3)
print(heights.shape[0])   # 2, the number of rows
print(heights.ndim)       # 2
print(heights.dtype)      # float64
```

`shape[0]` is the usual way to ask 'how many items are there?'. When something goes wrong, `print(x.shape)` first. Most numpy errors are shape errors.

2. Creating arrays

Code	Result
`np.array([1.0, 2.0, 3.0])`	array from a list
`np.zeros((5, 3))`	5 by 3 array of zeros
`np.ones((2, 4))`	2 by 4 array of ones
`np.zeros_like(a)`	zeros with the same shape and type as `a`

Use floats (`0.0`, not `0`) for measurements. An array made only of integers stays integer, which can silently truncate results.

3. Indexing, slicing and assigning

Indexing starts at 0. A comma separates one index per dimension, and `:` means all of that dimension.

```
a = np.arange(12).reshape(4, 3)
# [[ 0  1  2]
#  [ 3  4  5]
#  [ 6  7  8]
#  [ 9 10 11]]

a[0]        # [0 1 2]          first row
a[0, 2]     # 2                row 0, column 2
a[:, 2]     # [ 2  5  8 11]    column 2 of every row
a[1:3]      # rows 1 and 2     (end not included)
a[-1]       # [ 9 10 11]       last row
```

You can also assign into a part of an array, and the right-hand side must fit that part:

```
out = np.zeros((3, 2))
out[0] = [1.0, 2.0]    # fill row 0
out[2] = -out[0]       # fill row 2 from row 0
print(out)
# [[ 1.  2.]
#  [ 0.  0.]
#  [-1. -2.]]
```

Indexing a row gives you a vector you can do maths with, for example `a[0] - a[1]`.

4. Element-wise maths

Arithmetic applies to every element at once, so you rarely need a loop. A single number combined with an array is applied to every element.

```
x = np.array([1.0, 2.0, 3.0])
y = np.array([10.0, 20.0, 30.0])
k = 0.5

x + y             # [11. 22. 33.]
x * y             # [10. 40. 90.]   element-wise, not a dot product
x ** 2            # [1. 4. 9.]
2 * x - y + k * x ** 2   # build bigger expressions from the same pieces
x / 4.0           # [0.25 0.5 0.75]

np.sqrt(x)        # square root of each element
np.exp(-x)        # e to the power of minus each element
np.log(x)         # natural log of each element
```

`np.sqrt`, `np.exp` and `np.log` also work on a single number. Two arrays can be combined element by element only if their shapes match or can be broadcast (section 7).

Order of operations matters. `**` binds tighter than `*` and `/`, and a fractional power needs brackets:

```
27 ** (1 / 3)   # 3.0000000000000004   cube root
27 ** 1 / 3     # 9.0                  this means (27 ** 1) / 3, not a cube
root
```

Power is `**`, not `^` (`^` is bitwise XOR in Python).

In-place operators update the array they are applied to instead of making a new one:

```
z = np.array([1.0, 2.0, 3.0])
z += 1.0      # z is now [2. 3. 4.]
z *= 2        # z is now [4. 6. 8.]
```

5. Random numbers

```
rng = np.random.default_rng()

rng.normal(170, 8, size=5)       # 5 Gaussian numbers: mean 170, standard
deviation 8
rng.normal(0, 1, size=(4, 3))    # a 4 by 3 array of standard Gaussian
numbers
rng.uniform()                    # one number, uniform in [0, 1)
rng.uniform(-1, 1)               # one number, uniform in [-1, 1)

t = np.zeros((6, 2))
rng.normal(0, 1, size=t.shape)   # an array with exactly the shape of t
```

- `size=` takes a shape as a tuple. `size=t.shape` is the safe way to match another array.
- The second argument of `rng.normal` is the **standard deviation**, not the variance.
- A Gaussian number scales like this: `mean + std * rng.normal(0, 1)` has the same distribution as `rng.normal(mean, std)`.
- You will also see the older style `np.random.rand(4, 3)`, which takes the dimensions as separate arguments and gives uniform numbers in [0, 1).
- For repeatable results while debugging, seed the generator: `np.random.default_rng(42)`.

6. Sums, means and lengths

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])

np.sum(a)           # 21           everything
np.sum(a, axis=0)   # [5 7 9]      collapse the rows: one total per column
np.sum(a, axis=1)   # [ 6 15]      collapse the columns: one total per row
np.sum(a * a)       # 91           sum of the squares of every element
np.sum(a * a) / a.shape[0]   # 45.5  divide by the number of rows to get a
per-row average

np.linalg.norm(np.array([3.0, 4.0]))   # 5.0  the length of a vector
```

`axis=k` means that dimension disappears from the result's shape. A `(2, 3)` array summed over `axis=0` gives shape `(3,)`. Summing over `axis=2` of a `(5, 5, 3)` array gives shape `(5, 5)`.

`np.linalg.norm` on a single vector (1-D array) gives one number, its length. It needs a vector, so pick the row first, for example `np.linalg.norm(a[0] - a[1])`.

7. Broadcasting

When two shapes differ, numpy stretches the smaller one to fit, as long as the shapes agree when lined up from the right-hand end. A dimension of length 1 (or a missing one) is stretched.

```
grid = np.ones((4, 3))

grid * 2.0                          # a single number applies everywhere
grid + np.array([1.0, 2.0, 3.0])    # (4,3) + (3,): the vector is added to
every row

scale = np.array([1.0, 2.0, 3.0, 4.0])   # shape (4,)
# grid * scale                       # ERROR: (4,3) vs (4,) do not line up
grid * scale[:, None]                # OK: (4,3) * (4,1) scales row i by
scale[i]
```

If you get `operands could not be broadcast together`, write both shapes on paper and compare them from the right.

8. Every pair at once with `None`

Indexing with `None` (also written `np.newaxis`) inserts a new axis of length 1. Combined with broadcasting, this gives you a result for every combination of two sets without writing a double loop.

```
a = np.array([1, 2, 3])

a.shape              # (3,)
a[:, None].shape     # (3, 1)  a column
a[None, :].shape     # (1, 3)  a row

a[:, None] - a[None, :]
# [[ 0 -1 -2]
#  [ 1  0 -1]
#  [ 2  1  0]]       out[i, j] = a[i] - a[j]
```

The same idea works when every item is itself a row of numbers. Take 4 points with 2 coordinates each:

```
p = np.arange(8.0).reshape(4, 2)       # shape (4, 2)

d = p[:, None, :] - p[None, :, :]      # (4,1,2) - (1,4,2) -> (4,4,2)
# d[i, j] is the vector p[i] - p[j]

sq = np.sum(d * d, axis=2)             # (4,4): one squared length per pair
```

To multiply a table with one number per pair by those vectors, give the table a trailing axis so the shapes line up, then collapse an axis with `sum`:

```
w = np.ones((4, 4))                    # one weight per pair, shape (4, 4)
w[:, :, None].shape                    # (4, 4, 1)

weighted = w[:, :, None] * d           # (4,4,1) * (4,4,2) -> (4,4,2)
per_point = np.sum(weighted, axis=1)   # add up over j: shape (4, 2)
```

Check the shape after every line while you build up a calculation like this.

9. Remainders and rounding

```
np.mod(13, 12)        # 1    same as 13 % 12
np.mod(-2, 12)        # 10   the result is never negative for a positive
divisor
np.mod(np.array([13.0, 25.0, -2.0]), 12.0)   # [ 1.  1. 10.]  works on whole
arrays

np.round(3.7)         # 4.0
np.round(0.5)         # 0.0  halves round to the nearest EVEN number
```

These combine to give the offset from the nearest multiple of a step, which can be negative:

```
x = np.array([1.0, 4.0, 6.5, 9.0])
step = 10.0

x - step * np.round(x / step)   # [ 1.   4.  -3.5 -1. ]
np.mod(x, step)                 # [1.  4.  6.5 9. ]   offset from the
multiple below instead
```

The in-place version changes the array itself:

```
y = x.copy()
y -= step * np.round(y / step)   # y is modified; x is untouched because of
.copy()
```

10. Single numbers, comparisons and probabilities

A result such as `np.sum(...)` or `np.exp(...)` of single numbers is just a number, so you can use it in `if` statements.

```
energy_change = 2.3
scale = 0.8

if energy_change <= 0:            # a comparison gives True or False
    print('always accept')
else:
    p = np.exp(-energy_change / scale)   # a number between 0 and 1
    print('accept with probability', p)
```

To make something happen with probability `p`, draw a uniform number and compare it:

```
p = 0.3
happened = rng.uniform() <= p     # True about 30% of the time
```

`np.exp` of a large negative number is close to 0, and `np.exp(0)` is exactly 1. `np.log(1)` is 0.

11. Returning more than one value

A Python function can return several things at once as a tuple, and the caller unpacks them in order:

```
def describe(a):
    return np.mean(a), np.max(a)

m, biggest = describe(np.array([1.0, 5.0, 3.0]))   # m is 3.0, biggest is 5.0
```

The docstring tells you the order and what each value is. A function can have several `return` statements, and only the first one reached is used.

12. Gotchas that will bite you

1. **A function containing only `pass` returns `None`.** An error mentioning `NoneType` often just means you have not filled that function in yet.
2. **Assignment does not copy.** `b = a` makes both names point at the same array. Use `b = a.copy()` when you want an independent one.
3. **In-place operators and assignments change the original.** `a += 1` and `a[0] = 0` modify `a` itself. Be careful with arrays that were passed into a function.

4. **Operator precedence.** `x ** 1 / 3` is not a cube root. Use brackets: `x ** (1 / 3)`.
5. **Shape mismatch** (`could not be broadcast together`): print both `.shape`s and compare from the right.
6. **`*` is not a matrix product.** Use `@` for matrix multiplication.
7. **Integer arrays truncate.** Assigning `2.7` into an integer array stores `2`. Create arrays with floats.
8. **Keep track of units.** numpy does not know them. Write the unit in a comment next to every variable.
9. **Check what you return.** A function that is meant to give an array of shape `(N, 3)` should be tested with `print(result.shape)`.

13. Which sections help with which exercise

Exercise	Useful sections
1 Single atom	4
2 Free particles	4, 5, 9
3 Simple diatomic	3, 4, 6
4 Simple Lennard-Jones	4, 6, 7, 8, 9
5 Energies	4, 6, 8, 9
6 Temperature	4, 5
7 Pressure	4, 10, 11

Later exercises say 'copy from' an earlier one, so the sections you used before keep applying.

14. Try it yourself: a complete runnable script

Save as `playground.py`, run it, then change the numbers and see what happens.

```
import numpy as np

rng = np.random.default_rng(1)   # fixed seed: same numbers every run

# arrays and shapes
pts = rng.uniform(0, 10, size=(4, 2))   # 4 points with 2 coordinates each
print('shape        :', pts.shape)
```

```
# indexing, assigning into a row, zeros_like, norm
moves = np.zeros_like(pts)
moves[0] = pts[1] - pts[0]
print('length       :', np.linalg.norm(moves[0]))

# element-wise maths with scalars
noise = rng.normal(0, 0.5, size=pts.shape)
new_pts = pts + 2.0 * moves - 0.5 * noise
print('new_pts shape:', new_pts.shape)

# sums along an axis, and a per-row average
print('sum axis 0   :', np.sum(pts, axis=0))
print('sum axis 1   :', np.sum(pts, axis=1))
print('squares/row  :', np.sum(pts * pts) / pts.shape[0])

# None indexing: every pair, then collapse an axis
d = pts[:, None, :] - pts[None, :, :]
print('d shape      :', d.shape)
w = np.ones((4, 4))
print('collapsed    :', np.sum(w[:, :, None] * d, axis=1).shape)

# remainders and rounding
x = np.array([1.0, 4.0, 6.5, 9.0])
print('mod          :', np.mod(x, 5.0))
print('from nearest :', x - 10.0 * np.round(x / 10.0))

# copy versus alias, and an in-place operator
y = x.copy()
y -= 1.0
print('x:', x, ' y:', y)

# probabilities and returning two values
def describe(a):
    return np.mean(a), np.max(a)

m, biggest = describe(x)
happened = rng.uniform() <= 0.3
print(m, biggest, happened)
```